

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO4: Implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 15

Total Marks:30

Annexure A

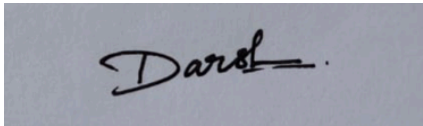
CLASS TEST

Code of Conduct:

I DARSHINIE KARANI.V.M (192520033) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:**CLASS TEST**

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Annexure A

CLASS TEST

QUESTIONS:

Total Marks: 30

Q1.A **Smart Healthcare Application** is being developed to manage patients, appointments, prescriptions, laboratory reports, and billing.

Question: Define architectural patterns in object-oriented systems. Identify any two common architectural patterns and explain how **Layered Architecture** can improve modularity, scalability, and maintainability in the healthcare application.

[L1, L2, L3]

Q2.An **Online Library System** stores information about books, members, borrowing transactions, reservations, and fines. Developers want to isolate database operations from the application logic.

Question: What is the **Repository Pattern**? Mention two advantages and explain how it can separate data-access operations from book-borrowing and reservation logic.

[L2, L3, L4]

Q3.A **Vehicle Rental Application** supports cars, bikes, vans, and electric vehicles. The system should allow new vehicle types to be introduced without breaking existing functionality.

Question: What are object-oriented design guidelines? Identify any two categories and explain how the **Open-Closed Principle (OCP)** supports extensibility and reusable components in the vehicle rental system.

[L2, L3, L4]

Q4.A **University Management System** stores student profiles, course registrations, attendance, examination results, and faculty information. These objects must remain available after the application is closed.

Question: Define **object persistence** in an object-oriented application. Mention two persistence mechanisms and explain how an **Object-Relational Mapping (ORM)** framework can manage persistent student and course objects.

[L1, L2, L3]

Q5. An **Online Travel Booking System** allows users to search flights and hotels, make reservations, process payments, and cancel bookings. Complex business rules must be coordinated without being placed directly inside domain entities.

Question: What is an **Application Layer**? State two major responsibilities and explain how it coordinates travel-booking use cases without directly implementing core domain business rules.

[L2, L3, L4]

Q6. An **Online Retail System** consists of a web/mobile interface, business services for orders and payments, and a database containing customer, product, and transaction information.

Question: Explain the structure of a **three-tier architecture** consisting of Presentation, Business, and Data layers. Analyze the flow of an online purchase from product selection through payment and order storage.

[L2, L4, L5]

Q7. A **Smart Parking Application** receives parking requests from mobile users and checks slot availability, calculates parking fees, and confirms reservations.

Question: What is the role of the **Business Logic Layer**? Identify two important functions and explain how it validates and processes parking requests received from the presentation layer.

[L2, L3, L4]

Q8. A **Smart Hospital System** contains separate modules for patient management, appointments, billing, pharmacy, and laboratory services. Some modules have become highly dependent on one another.

Question: Explain the significance of **cohesion and coupling** in object-oriented design. Analyze how high cohesion and low coupling can improve maintainability, reusability, and flexibility in the hospital system.

[L3, L4, L5]

Q9. A **Digital Banking Application** manages customer accounts and transactions. Account balances should not be modified directly by external classes.

Question: Define **encapsulation** in object-oriented systems. Explain how encapsulation protects account state and distinguish it from data hiding using a suitable banking example.

[L2, L4]

Q10.A **Smart E-Commerce Application** manages customers, products, shopping carts, orders, payments, inventory, and delivery services. The application must support a large number of users and protect sensitive transaction information.

Question: Design a suitable **layered object-oriented architecture** for the e-commerce application. Identify the major layers, recommend appropriate design patterns, and justify how the architecture improves security, scalability, maintainability, and flexibility.

[L3, L4, L5]

Q11.A **Smart Manufacturing System** communicates with databases, IoT sensors, payment services, external suppliers, and reporting systems. These technical operations should not be mixed with core manufacturing rules.

Question: What is the role of the **Infrastructure Layer** in object-oriented architecture? List two responsibilities and explain how it supports database access, IoT communication, and integration with external systems.

[L1, L2, L3]

Q12.A **Smart Traffic Management System** requires a centralized configuration manager to maintain common system settings used by multiple application components.

Question: Define the **Singleton Design Pattern**. Mention its major characteristics and explain one suitable use of Singleton for shared resources. Also discuss one limitation or risk associated with excessive use of the pattern.

[L2, L3, L4]

Q13.A **Smart Home Automation System** controls lights, fans, security cameras, thermostats, and smart locks. The application should work with different device implementations without directly depending on concrete device classes.

Question: What is **Dependency Injection** in object-oriented design? Mention two benefits and analyze how dependency injection reduces tight coupling and improves unit testing in the smart home system.

[L2, L3, L4]

Q14.An **Online Payment System** supports multiple payment strategies such as credit card, debit card, UPI, and digital wallet payments. The payment method may change according to customer selection.

Question: Explain the working of the **Strategy Design Pattern**. Compare the roles of **Context** and **Strategy**, and illustrate how the pattern can be applied to implement different payment methods.

[L3, L4, L5]

Q15.A **Smart Pharmacy Management System** manages customers, medicines, prescriptions, inventory, suppliers, billing, and online orders. The organization expects new services and medicine-management features to be added in the future.

Question: Design a suitable **layered architecture** for the pharmacy management system. Identify the major layers, recommend relevant design patterns, and justify how the proposed architecture supports flexibility, scalability, maintainability, and future extension.

[L3, L4, L5]

ANSWERS

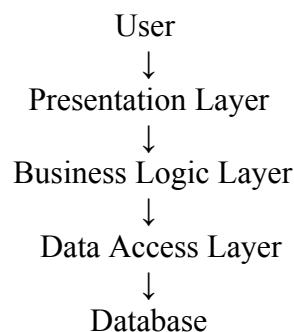
1.

Architectural patterns are reusable high-level structures used to organize the components of a software system and define how those components interact.

Two common architectural patterns are:

1. Layered Architecture – divides the system into separate layers, such as Presentation, Business Logic, and Data Access layers.
2. Client–Server Architecture – separates the client that requests services from the server that provides services and manages data.

Layered Architecture for a Smart Healthcare Application



- Presentation Layer: Provides interfaces for patients, doctors, and hospital staff.
- Business Logic Layer: Handles appointments, prescriptions, billing, and healthcare rules.
- Data Access Layer: Manages storage and retrieval of patient, appointment, and medical data.

Advantages

- Modularity: Each layer has a specific responsibility.
- Scalability: Individual layers can be improved or scaled independently.
- Maintainability: Changes in one layer have minimal effect on other layers.

Thus, layered architecture makes a smart healthcare system organized, maintainable, scalable, and easier to develop.

```

@startuml
left to right direction
actor Patient
actor Doctor
actor "Hospital Staff" as Staff

rectangle "Smart Healthcare System" {
    usecase "Manage Patient Information" as UC1
    usecase "Book Appointment" as UC2
    usecase "Manage Prescription" as UC3
    usecase "Generate Bill" as UC4
    usecase "Access Medical Records" as UC5
}

Patient --> UC1
Patient --> UC2
Patient --> UC5

Doctor --> UC3
Doctor --> UC4
Doctor --> UC5

Staff --> UC4
Staff --> UC5

```

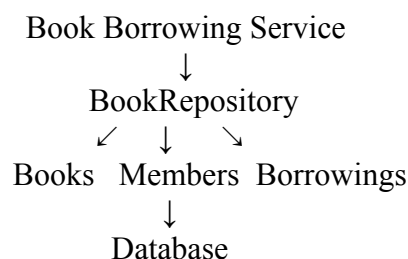
2.

The Repository Pattern is a design pattern that provides an abstraction between the business logic and data-access layer. It allows the application to access data through repository methods without directly dealing with database operations.

Advantages:

1. Separation of concerns: Database operations are separated from business logic.
2. Maintainability: Changes in the database do not require major changes to the business logic.
3. Testability: Repositories can be replaced with mock objects during testing.
4. Reusability: Common data-access operations can be reused by different parts of the application.

Example – Online Library System:



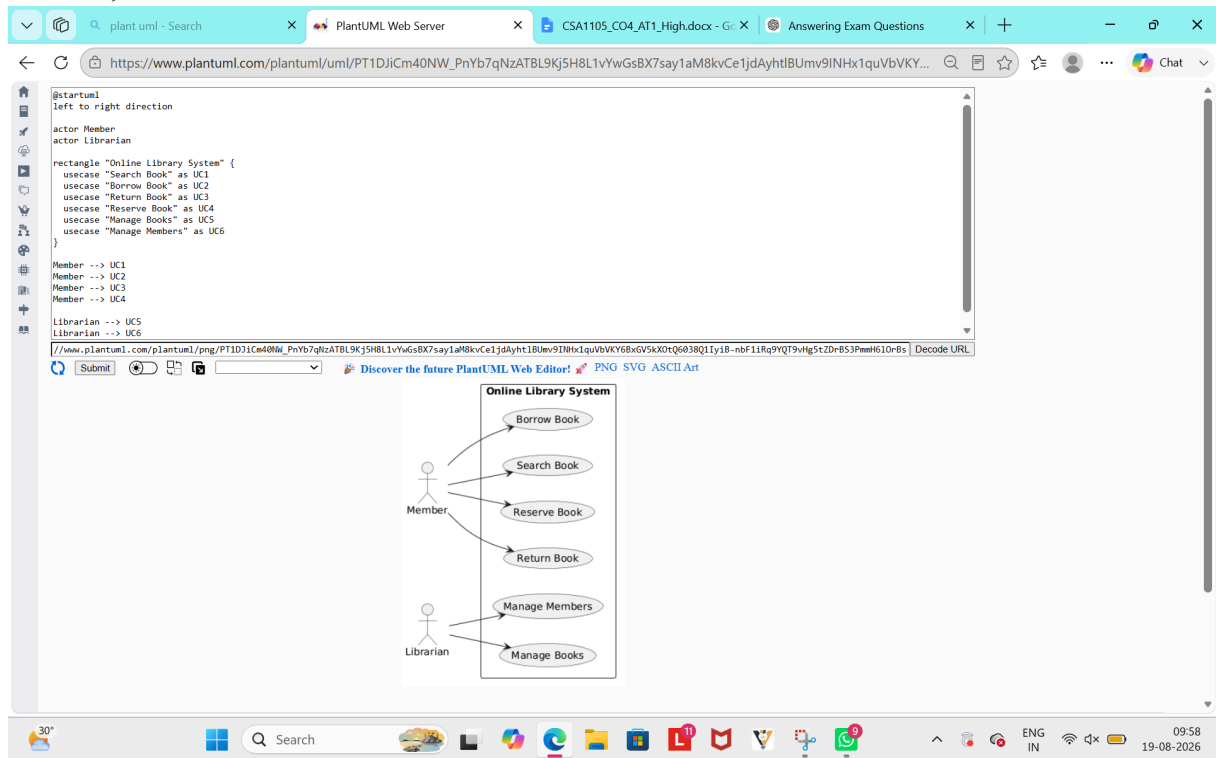
The BookRepository can provide methods such as:

- findBook()
- saveBook()

- findMember()
- saveBorrowing()
- findReservation()

For example, when a student borrows a book, the Book Borrowing Service calls saveBorrowing() instead of directly accessing the database.

Thus, the Repository Pattern makes the online library system loosely coupled, maintainable, reusable, and easier to test.



3.

Object-oriented design guidelines are principles used to design software that is maintainable, reusable, flexible, extensible, and loosely coupled.

Two important categories of object-oriented design principles are:

1. SOLID principles
2. GRASP principles

Open-Closed Principle (OCP)

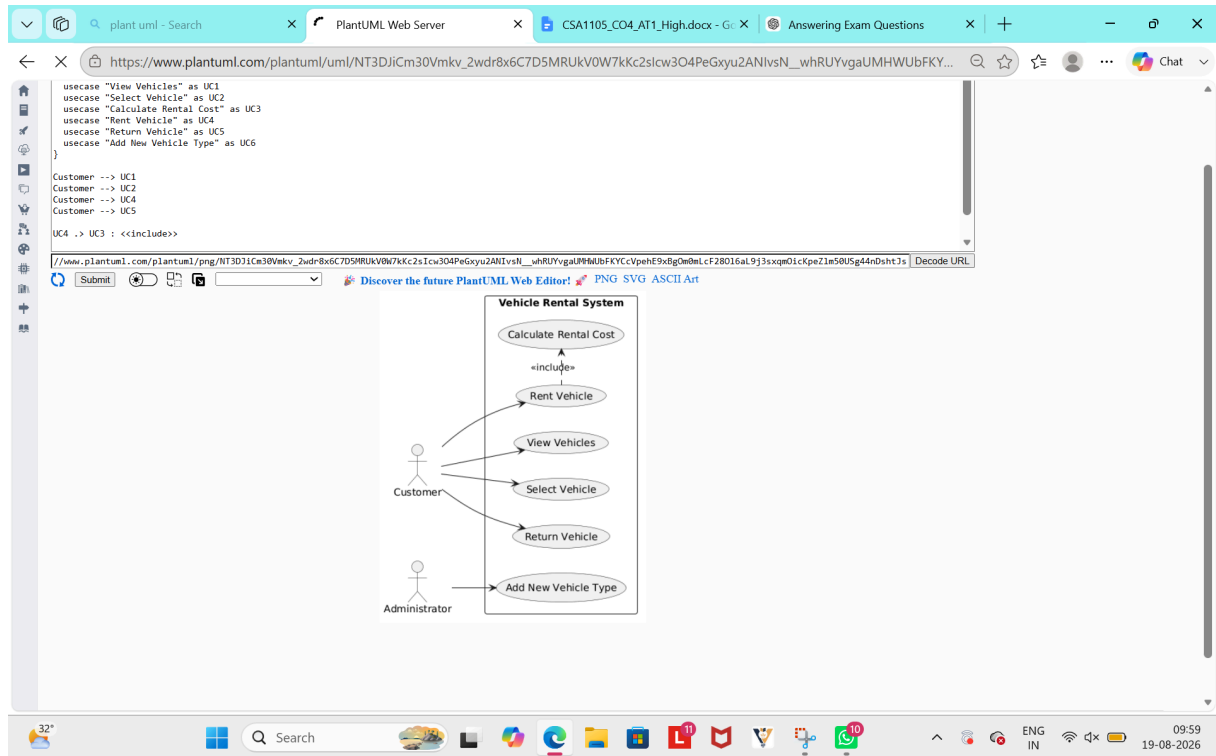
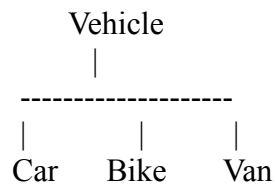
The Open-Closed Principle is one of the SOLID principles. It states that:

Software entities should be open for extension but closed for modification.

This means that we should be able to add new functionality without changing the existing, tested code.

Example: Vehicle Rental System

Consider a vehicle rental system that supports different types of vehicles such as:



Benefits of OCP

- Easy extension: New vehicle types can be added easily.
- Reduced risk: Existing working code is not changed unnecessarily.
- Better maintainability: Each vehicle class manages its own rental calculation.
- Reusability: The common **Vehicle** interface can be used throughout the system.
- Flexibility: The system can support future vehicle types without major redesign.

Conclusion: By applying the Open-Closed Principle, the vehicle rental system becomes flexible, maintainable, and easier to extend while minimizing changes to existing code.

4.

Object persistence is the ability to store objects permanently so that the data remains available even after the application is closed or restarted.

Persistence Mechanisms

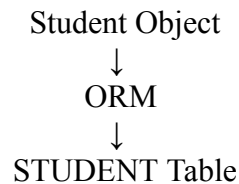
Two common mechanisms for object persistence are:

1. Object Database

- Stores objects directly in an object-oriented database.
 - Objects can be retrieved with their properties and relationships.
2. Relational Database with ORM
- Stores object data in relational database tables.
 - ORM (Object-Relational Mapping) maps classes and objects to tables and rows.

ORM Example : Student Registration System

Consider a system containing Student, Course, Registration, and Attendance objects.



For example:

Student Class

```

-----
studentId
studentName
course
  
```

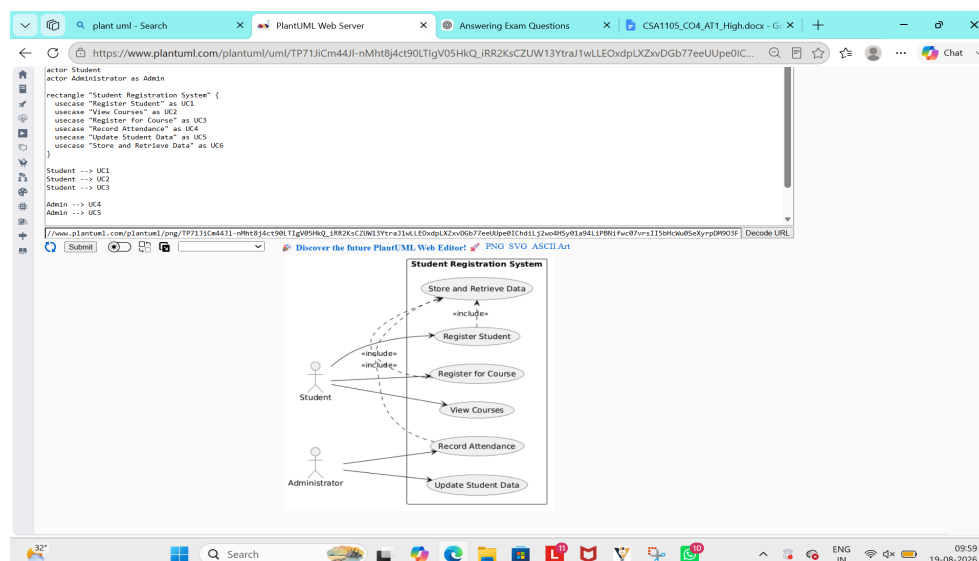
can be mapped to:

STUDENT Table

```

-----
student_id
student_name
course
  
```

Similarly, Course, Registration, and Attendance objects can be mapped to their corresponding database tables.



Advantages of ORM

- Reduces SQL coding: Developers can work with objects instead of writing SQL for every operation.
- Improves maintainability: Database operations are separated from application logic.
- Supports object relationships: Relationships between objects can be represented through database relationships.
- Simplifies data access: Objects can be easily stored, retrieved, updated, and deleted.

Conclusion:

Object persistence ensures that application objects are stored permanently, while ORM provides an easy way to map object-oriented classes to relational database tables.

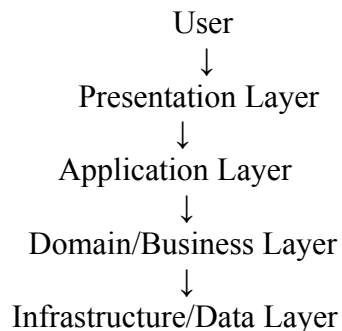
5.

The Application Layer is responsible for coordinating the application's use cases and workflows. It acts as a bridge between the Presentation Layer and the Domain/Business Layer.

Major Responsibilities

1. Coordinates application workflows:
It controls the sequence of operations required to complete a use case.
2. Manages communication between layers:
It receives requests from the Presentation Layer and coordinates the required domain objects and services.

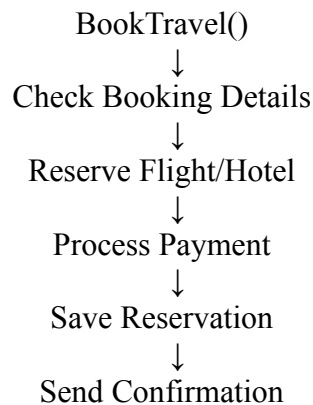
Example – Online Travel System



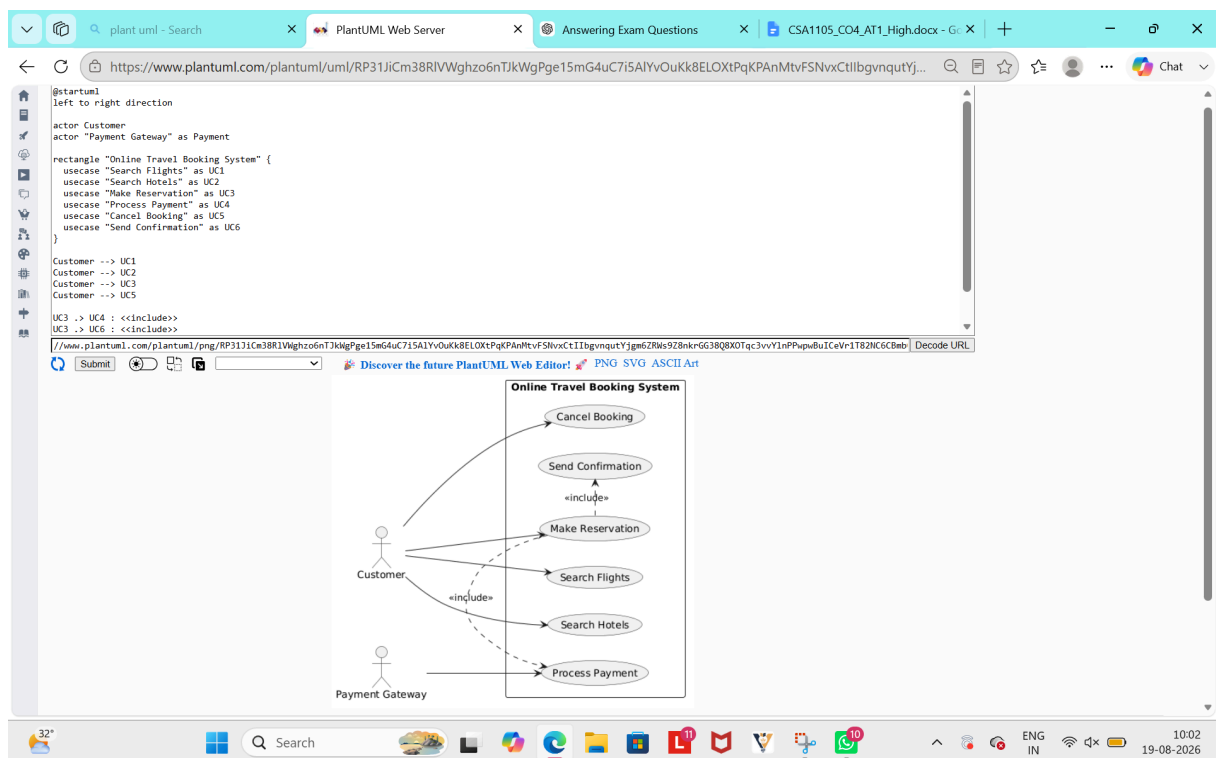
Suppose a user wants to book a trip. The Application Layer performs the following steps:

1. Receives the booking request from the user interface.
2. Coordinates the flight or hotel booking process.
3. Calls the required domain services.
4. Coordinates payment processing.
5. Stores the booking information.
6. Returns the booking confirmation to the user.

For example:



The Application Layer coordinates these operations, while the actual business rules, such as fare calculation or cancellation policies, remain in the Domain/Business Layer.



Conclusion:

The Application Layer helps maintain a clear separation between the user interface and business logic, making the system organized, maintainable, and easier to manage.

6.

Three-Tier Architecture

A three-tier architecture divides an application into three separate layers:

1. Presentation Layer
2. Business Layer

3. Data Layer

This separation improves maintainability, scalability, security, and modularity.

Presentation Layer

The Presentation Layer provides the interface through which customers interact with the online retail system.

Functions:

- Displays products and product details.
- Allows users to select products and add them to the cart.
- Collects customer and payment information.
- Displays order and payment confirmation.

Examples include the web application or mobile application.

Business Layer

The Business Layer contains the application services and business processes.

Functions:

- Validates customer orders.
- Checks product availability.
- Calculates the total price.
- Processes payment.
- Creates and manages orders.

For example, an OrderService coordinates order processing, while a PaymentService handles payment-related operations.

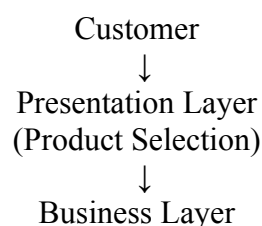
Data Layer

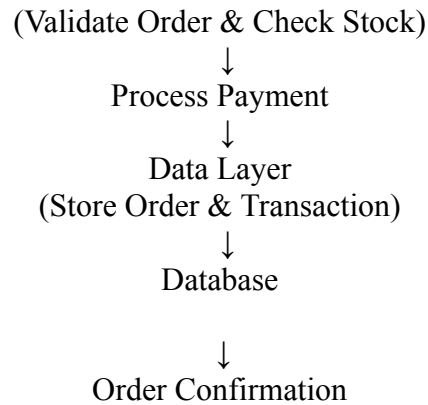
The Data Layer is responsible for storing and retrieving data from the database.

It manages information such as:

- Customer details
- Product information
- Orders
- Payment and transaction records

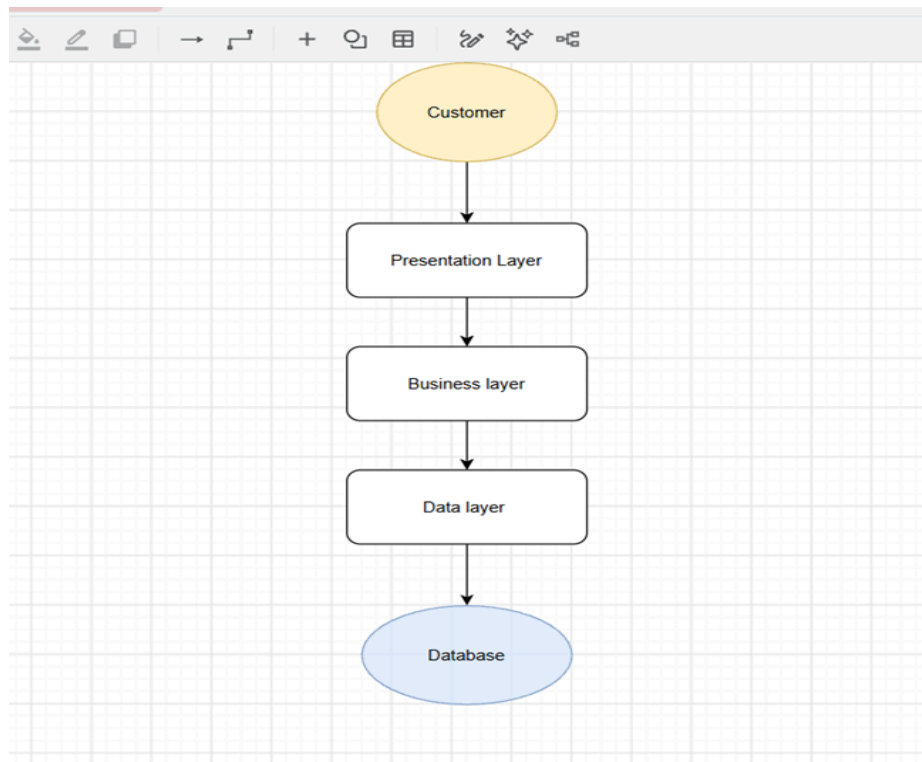
Flow of an Online Purchase





Step-by-step flow:

1. The customer selects a product using the web or mobile interface in the Presentation Layer.
2. The request is sent to the Business Layer.
3. The Business Layer checks product availability, validates the order, and calculates the total amount.
4. The payment is processed through the appropriate payment service.
5. After successful payment, the Business Layer sends the order and transaction details to the Data Layer.
6. The Data Layer stores the customer, order, product, and transaction information in the database.
7. Finally, an order confirmation is sent back to the Presentation Layer and displayed to the customer.



Conclusion:

Three-tier architecture clearly separates the user interface, business processing, and data management. This makes the online retail system easier to maintain, scale, test, and secure.

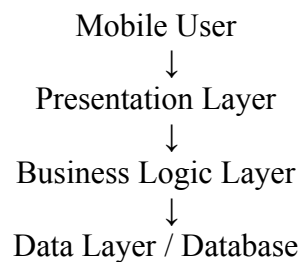
7.

The Business Logic Layer (BLL) is responsible for processing requests and implementing the application's business rules. It acts as a bridge between the Presentation Layer and the Data Layer.

Two Important Functions

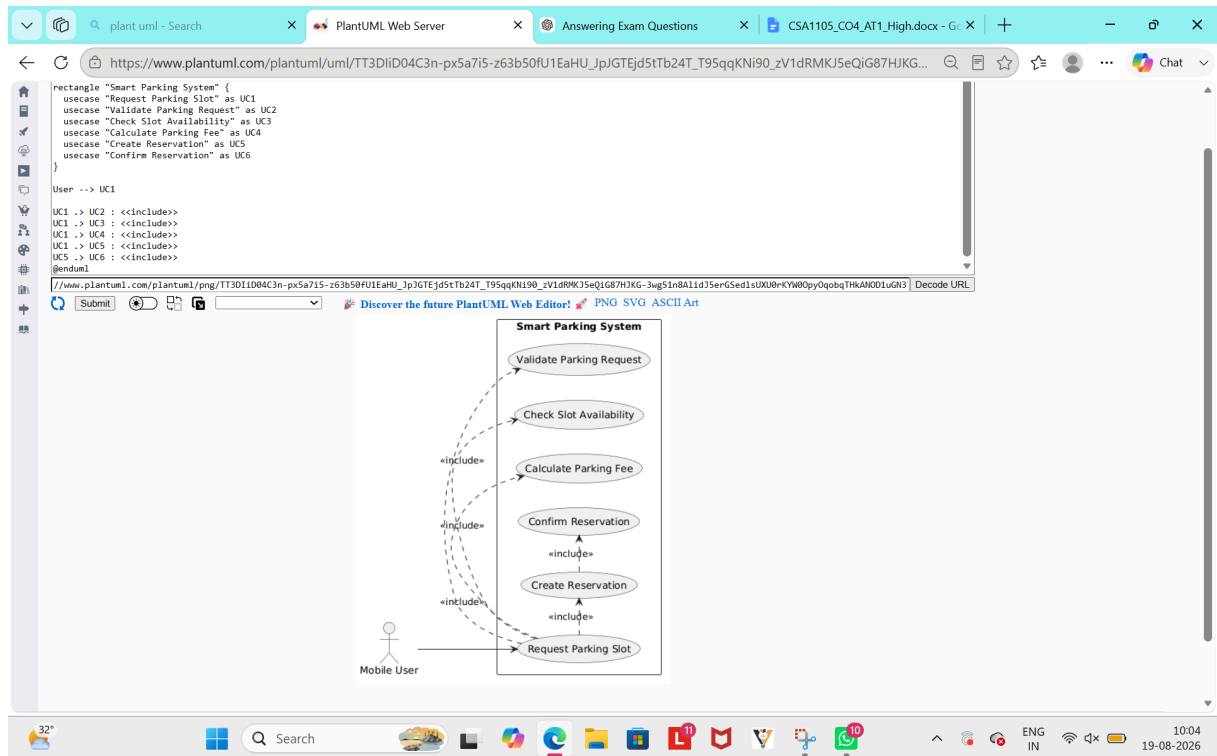
1. Validation of requests:
It checks whether the information provided by the user is valid before processing the request.
2. Processing business rules:
It applies rules such as checking parking-slot availability, calculating parking fees, and creating reservations.

Processing a Smart Parking Request



When a mobile user requests a parking slot, the process is as follows:

1. The Presentation Layer sends the parking request to the Business Logic Layer.
2. The BLL validates the request, such as checking the user's details and requested parking information.
3. It checks whether the requested parking slot is available.
4. If a slot is available, the BLL applies the required business rules and calculates the parking fee.
5. It creates and processes the parking reservation.
6. The reservation details are sent to the Data Layer for storage in the database.
7. Finally, the confirmation is returned to the Presentation Layer and displayed to the mobile user.



Conclusion

The Business Logic Layer ensures that parking requests are properly validated and processed according to business rules, while keeping the user interface separate from the application's core processing logic.

8.

Cohesion

Cohesion refers to how closely related the responsibilities and functions within a class or module are.

A module has high cohesion when it performs a specific and well-defined task.

For example, in a Smart Hospital System:

- Patient Management Module → manages patient information.
- Appointment Module → handles appointment scheduling.
- Billing Module → manages bills and payments.
- Pharmacy Module → handles medicines and prescriptions.
- Laboratory Module → manages laboratory tests and reports.

Each module should focus mainly on its own responsibility.

Coupling

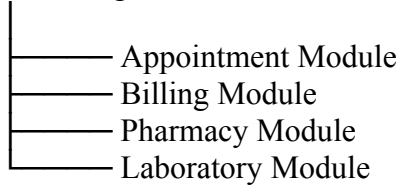
Coupling refers to the degree of dependency between different modules or classes.

Low coupling means that modules have fewer dependencies on one another. A change in one module should have minimal impact on other modules.

Application in the Smart Hospital System

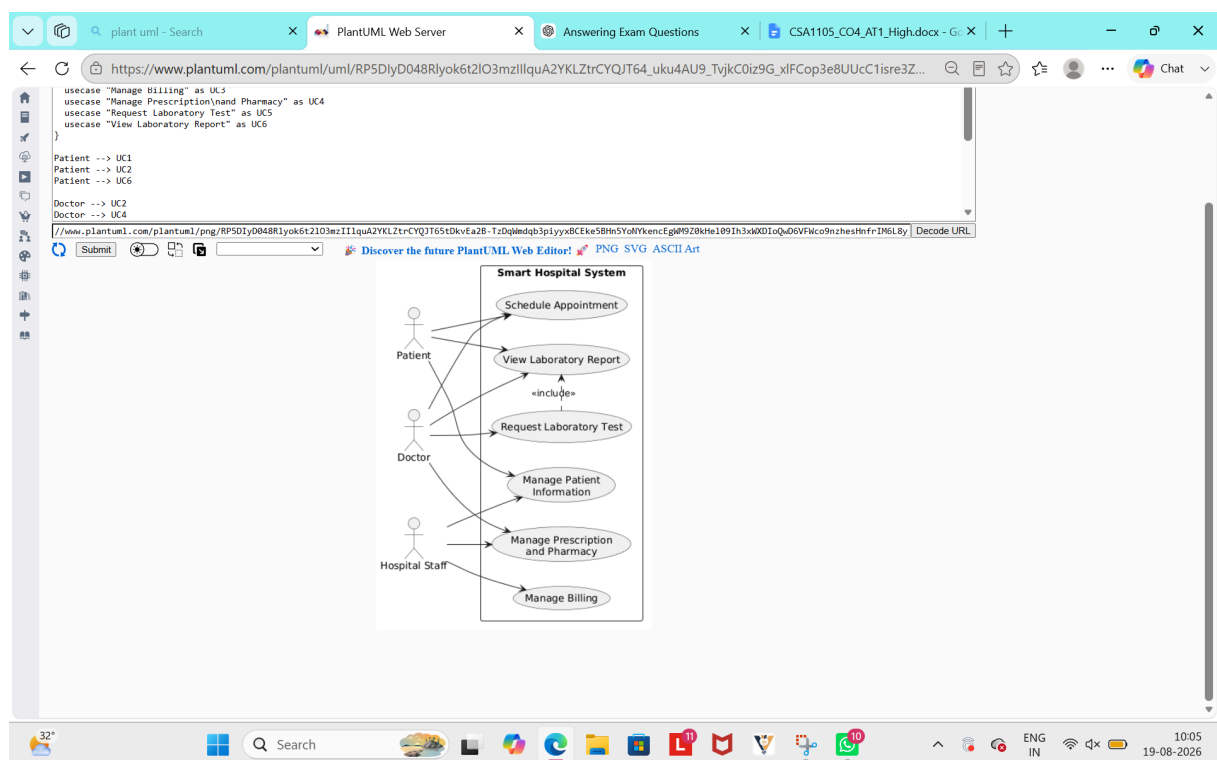
A good design would have the following structure:

Patient Management



Instead of modules directly depending heavily on each other's internal implementation, they should communicate through well-defined interfaces or services.

For example, if the Billing Module needs patient information, it should request only the required information through an interface rather than directly accessing the internal data and functions of the Patient Management Module.



Benefits of High Cohesion and Low Coupling

1. Improved Maintainability

Changes made to the Pharmacy Module, for example, will have minimal effect on the Appointment or Laboratory modules.

2. Better Reusability

A highly cohesive module, such as the Billing Module, can be reused in another healthcare application with fewer changes.

3. Greater Flexibility

New services or modules can be added easily without significantly modifying existing modules.

4. Easy Testing and Debugging

Since each module has a clear responsibility and limited dependencies, modules can be tested independently.

Conclusion

A well-designed Smart Hospital System should aim for high cohesion and low coupling. This makes the system more maintainable, reusable, flexible, and easier to test and extend.

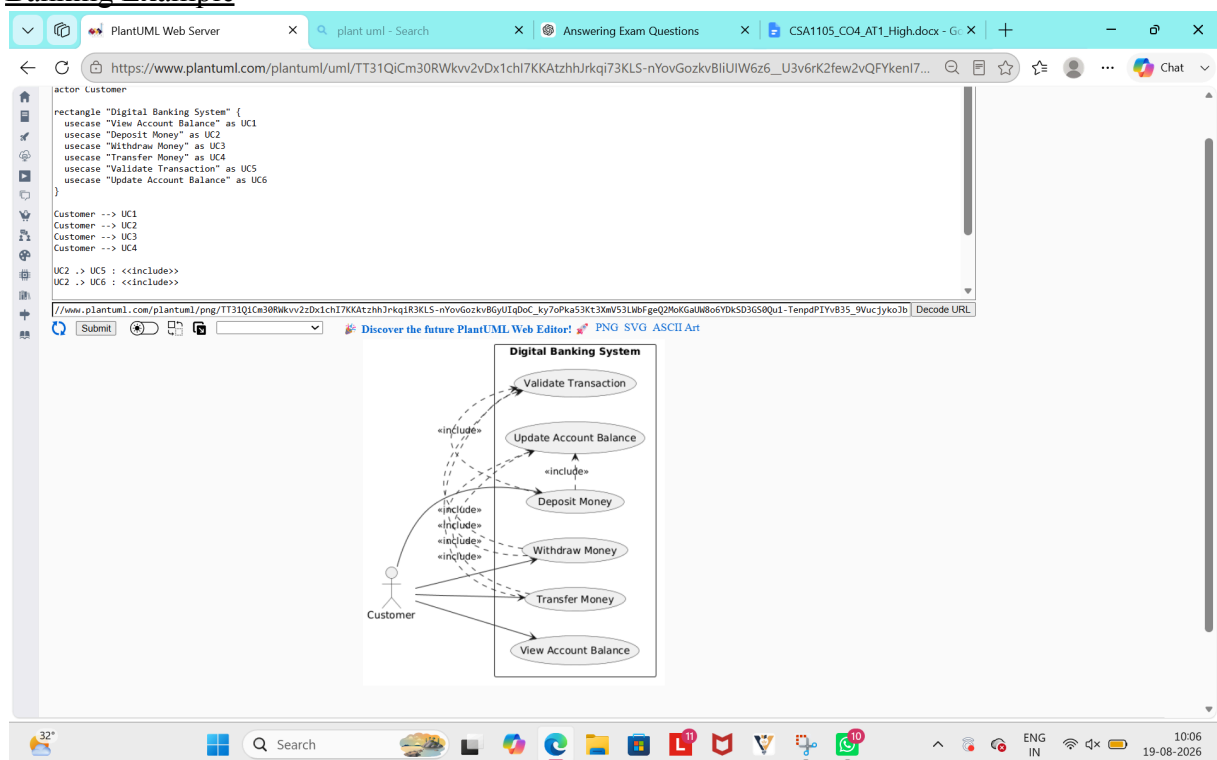
9.

Encapsulation

Encapsulation is an object-oriented principle in which data and the methods that operate on that data are combined within a single class. It also controls how the internal data of an object can be accessed or modified.

In a Digital Banking Application, the account balance should not be directly changed by external classes.

Banking Example



How Encapsulation Protects Account State

Encapsulation protects the account state by:

- Preventing unauthorized or invalid direct changes to the balance.
- Ensuring that changes occur only through controlled methods.

- Applying validation rules before updating the account.
- Maintaining the consistency and security of account information.

Encapsulation vs Data Hiding

Encapsulation	Data Hiding
Combines data and methods into a single class.	Restricts direct access to internal data.
Focuses on bundling data with related behavior.	Focuses on protecting the internal state.
Provides controlled access to an object's data.	Usually achieved using access modifiers such as private.

Conclusion

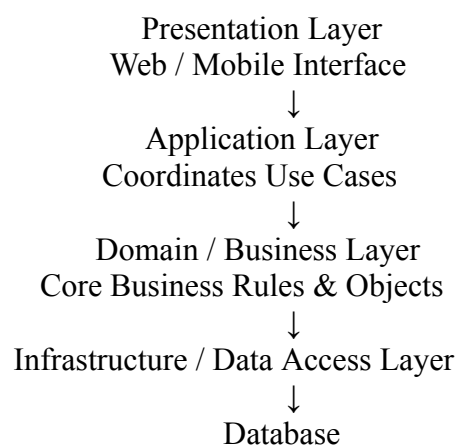
In the banking application, encapsulation groups the account balance and transaction methods inside the BankAccount class, while data hiding protects the balance by making it private. Together, they ensure that the account balance can only be modified through valid and controlled operations.

10.

Layered Object-Oriented Architecture for Smart E-Commerce

A suitable architecture for the Smart E-Commerce Application is a layered architecture, where each layer has a specific responsibility. This separation helps manage customers, products, carts, orders, payments, inventory, and delivery services efficiently.

Major Layers



Presentation Layer

This layer provides the web and mobile user interfaces.

Responsibilities:

- Customer registration and login.
- Product browsing and searching.
- Shopping cart management.
- Displaying orders and payment information.

Application Layer

This layer coordinates application use cases and workflows.

For example, when a customer places an order, it coordinates:

- Order creation.
- Inventory checking.
- Payment processing.
- Delivery arrangement.
- Order confirmation.

Domain/Business Layer

This layer contains the core business objects and business rules.

Main objects may include:

- Customer
- Product
- ShoppingCart
- Order
- Payment
- Inventory
- Delivery

Examples of business rules:

- Product availability.
- Order validation.
- Price calculation.
- Inventory updates.
- Delivery rules.

Infrastructure/Data Access Layer

This layer handles technical services such as:

- Database access.
- Data storage and retrieval.
- External payment gateways.
- Notification services.
- Delivery-service integration.

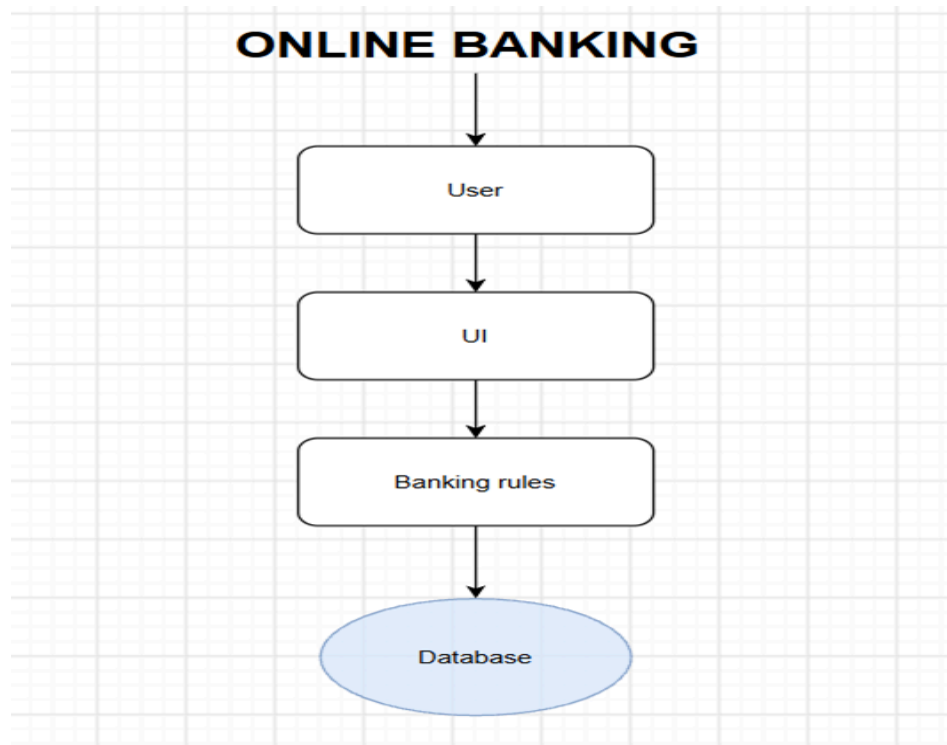
Recommended Design Patterns

- Repository Pattern
- Used to separate business logic from database operations.

Examples:

- CustomerRepository
- ProductRepository
- OrderRepository

This improves maintainability and testability.



Strategy Pattern

Used when different algorithms or methods can be selected at runtime.

For example:

```

PaymentStrategy
├── CreditCardPayment
├── DebitCardPayment
├── UPI/OnlinePayment
└── WalletPayment
  
```

It allows new payment methods to be added easily.

Factory Pattern

Used to create objects without exposing the complete object-creation logic.

For example, a factory can create different types of:

- Payment objects.
- Delivery services.
- Product-related objects.

Dependency Injection

Dependencies are provided to classes from outside rather than being created internally. This reduces tight coupling and improves testing and flexibility.

How the Architecture Improves Quality

Security

Sensitive information such as payment and transaction data can be handled through controlled business and infrastructure layers. The presentation layer does not directly access the database.

Scalability

Different layers or services can be scaled independently. For example, the web interface or payment service can handle increased user traffic without redesigning the entire system.

Maintainability

Each layer has a clear responsibility. Changes to the database or user interface have less impact on the core business logic.

Flexibility

New payment methods, delivery services, or product features can be added using patterns such as Strategy, Factory, and Dependency Injection without major changes to existing code.

Conclusion

Thus, a layered object-oriented architecture consisting of Presentation, Application, Domain/Business, and Infrastructure/Data layers, combined with suitable design patterns, provides a secure, scalable, maintainable, and flexible Smart E-Commerce Application.